

Capa de Transporte

Libro de Kurose

- “Un protocolo de la capa de transporte proporciona una comunicación lógica entre procesos de aplicación que se ejecutan en hosts diferentes”
- “Utilizando técnicas de control de flujo, números de secuencia, mensajes de reconocimiento y temporizadores (técnicas que estudiaremos en detalle en este capítulo), TCP garantiza que los datos transmitidos por el proceso emisor sean entregados al proceso receptor, correctamente y en orden. De este modo, TCP convierte el servicio no fiable de IP entre sistemas terminales en un servicio de transporte de datos fiable entre procesos. TCP también proporciona mecanismos de control de congestión. El control de congestión no es tanto un servicio proporcionado a la aplicación invocante, cuanto un servicio que se presta a Internet como un todo, un servicio para el bien común. En otras palabras, los mecanismos de **control de congestión** de TCP evitan que cualquier conexión TCP inunde con una cantidad de tráfico excesiva los enlaces y routers existentes entre los hosts que están comunicándose. TCP se esfuerza en proporcionar a cada conexión que atraviesa un enlace congestionado la misma cuota de ancho de banda del enlace. Esto se consigue regulando la velocidad a la que los lados emisores de las conexiones TCP pueden enviar tráfico a la red.” [p. 157]

Protocolos de transferencia de datos fiables

- “Las sumas de comprobación, los números de secuencia, los temporizadores y los paquetes de reconocimiento positivo y negativo desempeñan un papel fundamental y necesario en el funcionamiento del protocolo.” [p. 177]
- “El uso que hace TCP de los **números de secuencia** refleja este punto de vista, en el sentido de que los números de secuencia hacen referencia al **flujo de bytes** transmitido y no a la **serie de segmentos** transmitidos” [p. 195]

Sobre como se utilizan los números de secuencia y de reconocimiento en la comunicación TCP

- “El **número de secuencia** identifica al **primer byte del flujo de datos**, que se envía entre el TCP emisor al TCP remoto, que contiene el segmento. Si tenemos en cuenta que el flujo de byte fluye en una dirección entre las dos aplicaciones, TCP enumera cada byte con un número de secuencia. [...] Como cada byte que se manda es enumerado, el **número de reconocimiento** (número ACK) **contiene el próximo número de secuencia** (el próximo byte) **que quien envía el reconocimiento espera recibir**. Este es por lo tanto el número de secuencia del último byte recibido con éxito sumado 1 (el próximo en el flujo). Este campo es válido solo si el flag ACK está activado, que por lo general es lo que pasa a menos que el segmento sea uno de inicio o de cierre.” [Stevens, p. 588]

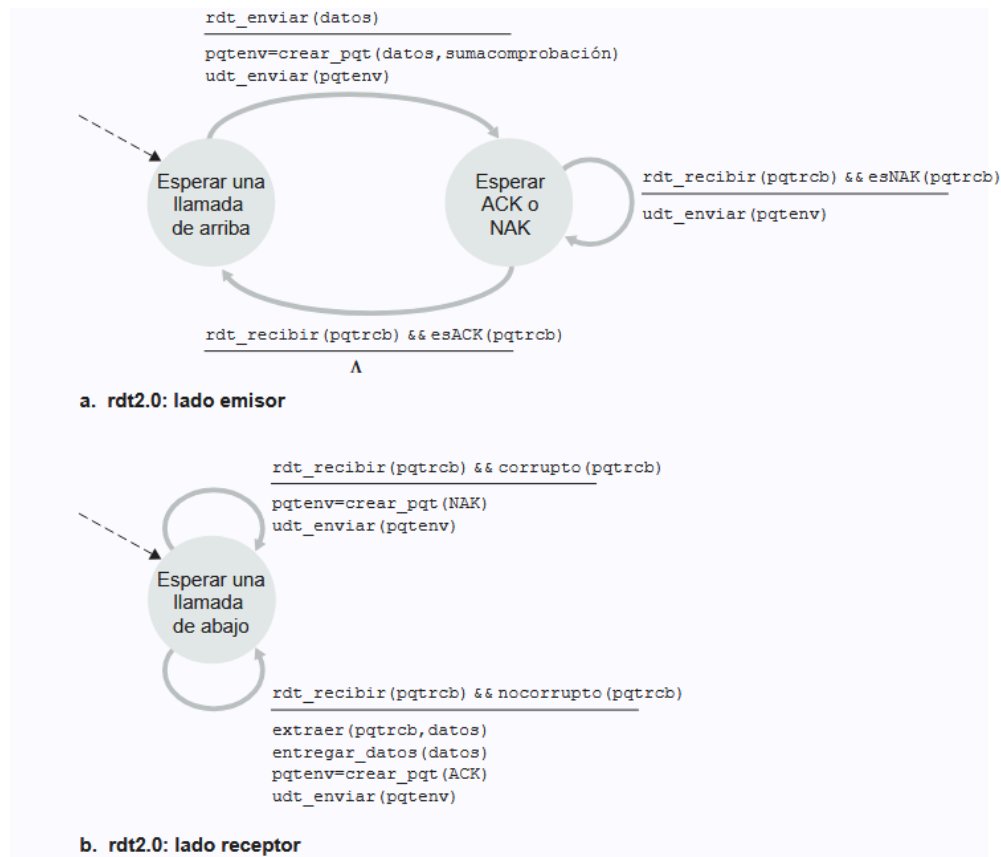
Sobre como hace un emisor para detectar la pérdida de paquetes

- Se podría esperar un tiempo que se acerque al que tardaría un paquete en ir y volver idealmente → pero esto ralentizaría mucho la operación → entonces, en la práctica “el emisor selecciona juiciosamente un intervalo de tiempo tal que sea probable que un paquete se haya perdido, aunque no sea seguro que tal pérdida se haya producido” [p. 177] → ¿y si el paquete no se perdió, si no que solo se demoró mas de lo previsto? → esto lo soluciona el número de secuencia (ISN)

Sobre las formas de transferencia de datos que se pueden implementar

Protocolo Stop and Wait (S&W)

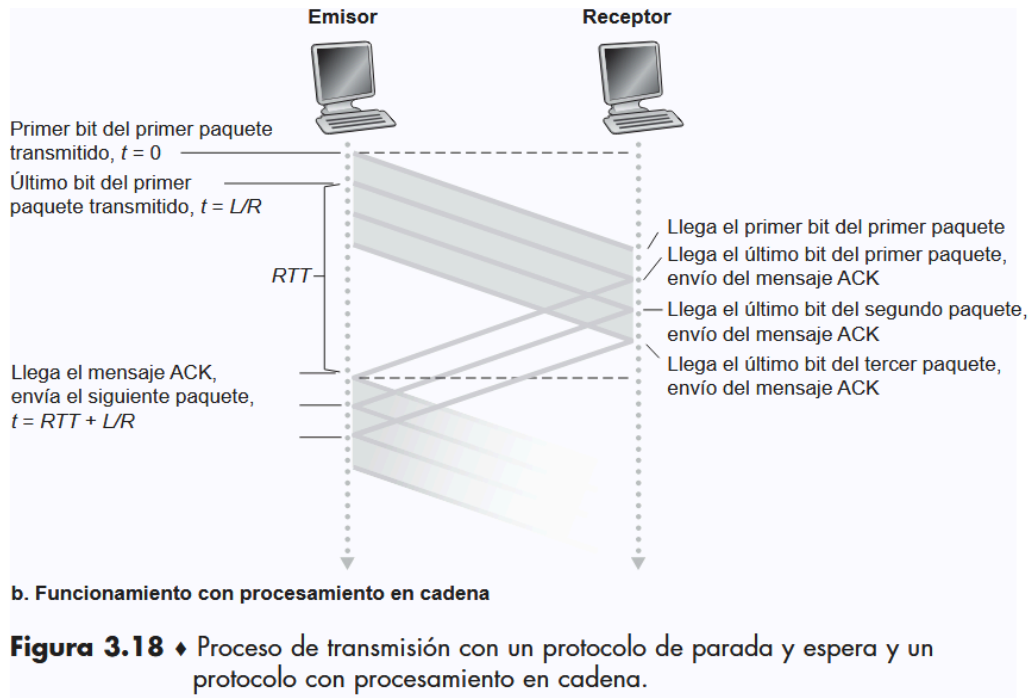
- En un protocolo de transporte que implementa esta forma de transferir datos, el emisor enviará un paquete, y no enviará otro hasta que le llegué una confirmación por parte del receptor de que el mensaje llegó sano y salvo. Si el paquete no llega, el emisor lo envía las veces que sean necesarias.



Representación gráfica de Kurose.

- El problema principal de este protocolo es que es lento → para ver un ejemplo de esto, mirar el ejemplo de las páginas 178 - 180

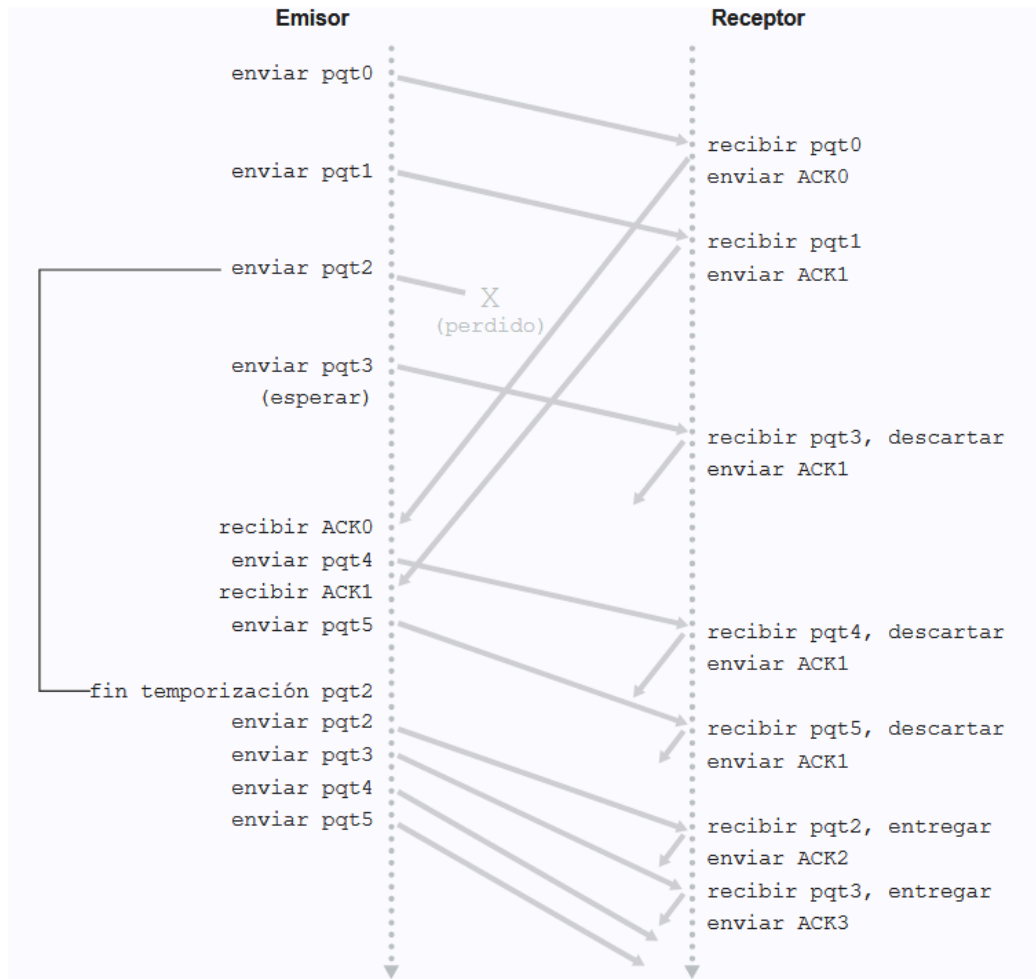
Protocolo pipelining



- Le permite a los emisores, y también a los receptores, enviar segmentos sin esperar por la confirmación del otro.
- Requiere incrementar el rango de los números de secuencia, requiere contar con buffers que puedan recibir mas de un segmento, y con buffers que mantengan del lado del emisor los paquetes que fueron enviados pero aún no reconocidos

Protocolo Go-back-N

Protocolo de "ventana deslizante". La idea principal de esta forma de transmisión es la de colocar un límite para los segmentos que se pueden enviar. Dicho límite se coloca en base a la cantidad de segmentos no reconocidos (es decir, que no se recibió el segmento de respuesta con ACK correspondiente) que hayan en ese determinado momento en la ventana.



- El emisor puede transmitir varios paquetes sin tener que esperar a que sean reconocidos, pero está restringido a no tener más de un número máximo permitido N de paquetes no reconocidos en el canal.
- Introduce el **tamaño de ventana** de segmentos retenibles en un momento dado

Explicación con algunos diagramas

- **Sobre los paquetes que se van recibiendo**

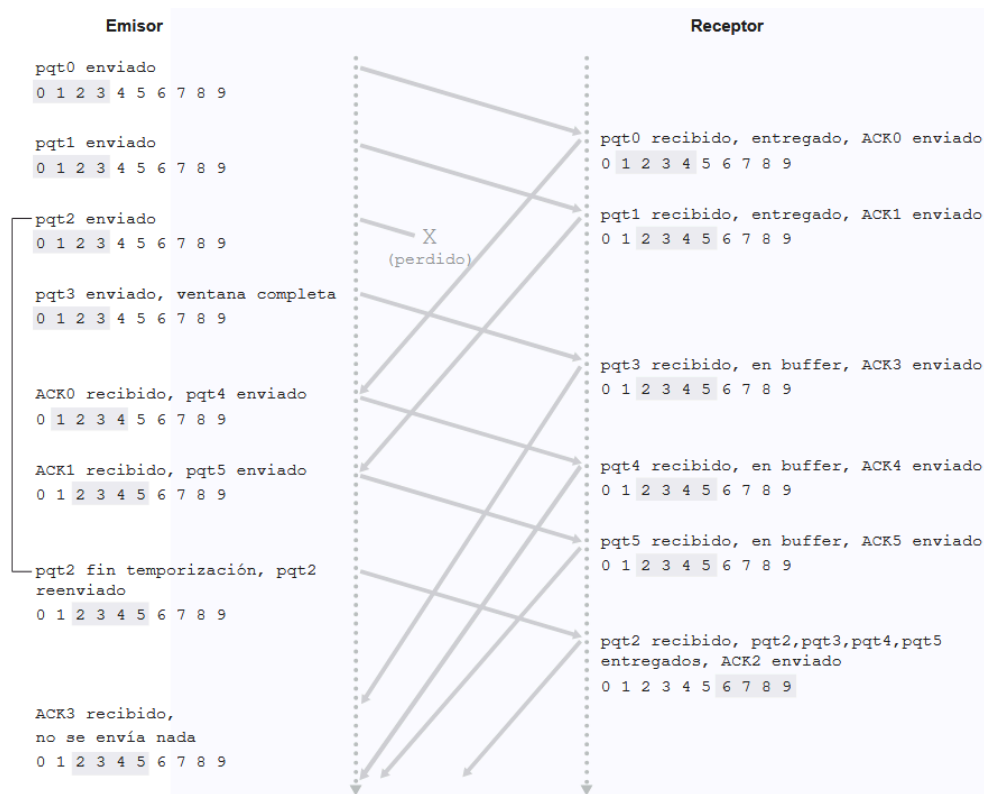
"Si un paquete con un número de secuencia N se recibe correctamente y en orden (es decir, los últimos datos entregados a la capa superior proceden de un paquete con el número de secuencia $N - 1$), el receptor envía un paquete ACK para el paquete N y entrega la parte de los datos del paquete a la capa superior. En todos los restantes casos, el receptor descarta el paquete y reenvía un mensaje ACK para el paquete recibido en orden más recientemente. Observe que dado que los paquetes se entregan a la capa superior de uno en uno, si el paquete K ha sido recibido y entregado, entonces todos los paquetes con un número de secuencia menor que K también han sido entregados"

- En nuestro protocolo GBN, el receptor descarta los paquetes que no están en orden (por mas que hayan sido recibidos sin pérdida) → La ventaja de este método es la simplicidad del almacenamiento en el buffer del receptor (el receptor no necesita almacenar en el buffer ninguno de los paquetes entregados desordenados)

- Lo que hace esto es que, en una **relación emisor - receptor** cualquiera, sea el emisor el encargado de controlar y saber el estado de sus paquetes y cuales tiene que reenviar y cuales no, abstrayendo de todo esto al receptor, quien solo tiene que mantener el número de secuencia del siguiente paquete en orden.
- **Desventajas** → Si el tamaño de la ventana fuera grande, y el primero de una serie larga de segmentos enviados se perdiera, o tardara mucho en llegar, habría que reenviar todos los paquetes a partir del primero que falló.

Protocolo Repetición-Selectiva

- Busca solucionar la desventaja que tiene el protocolo anterior permitiendo que se puedan **reconocer** y reenviar paquetes individuales, de todos aquellos que se sospeche que llegaron corrompidos o que se perdieron.



- "El receptor de SR confirmará que un paquete se ha recibido correctamente tanto si se ha recibido en el orden correcto como si no. Los paquetes no recibidos en orden se almacenarán en el buffer hasta que se reciban los paquetes que faltan (es decir, los paquetes con números de secuencia menores), momento en el que un lote de paquetes puede entregarse en orden a la capa superior" [p. 186]
- A diferencia de los otros protocolos, utiliza un **reconocimiento individual** en vez de **acumulativo**
- Desde **el lado del emisor, cada paquete debe tener su propio temporizador lógico**, ya que solo se transmitirá un paquete al producirse el fin de la temporización. **Si se ha recibido un mensaje ACK, el emisor de SR marca dicho paquete como que ha sido recibido**, siempre que esté dentro de la ventana. Si el número de secuencia del paquete es **igual a base_emision, se hace avanzar la base de la ventana**, situándola en el paquete no reconocido que tenga el número de secuencia más bajo.
- Desde **el lado del receptor se implementa una ventana** para los paquetes enviados por el emisor.

- Desde **el lado del receptor**, salvo el caso en el que se haya recibido correctamente un paquete cuyo número de secuencia pertenezca al intervalo $[base_recepcion, base_recepcion+N-1]$, y el caso en el que se haya recibido correctamente un paquete cuyo número de secuencia pertenece al intervalo $[base_recepcion-N, base_recepcion-1]$, se ignoran los paquetes
- Una cosa importante para subrayar es que “El emisor y el receptor no siempre tienen una visión idéntica de lo que se ha recibido correctamente y de lo que no. En los protocolos SR, esto significa que **las ventanas del emisor y del receptor no siempre coinciden**”
- **Desventajas** → Surje una complicación cuando el receptor tiene que decidir si un paquete enviado con un mismo número de secuencia es un paquete previamente recibido pero ahora retransmitido (ya corrió el RTO), o si dicho paquete es uno nuevo y distinto. Como se encuentra implementada la comunicación, el receptor no puede conocer esta información. [Para un ejemplo de esto, ir a las páginas 187 - 190 del libro.](#) → El tamaño de la ventana tiene que ser menor o igual que la mitad del tamaño del espacio de números de secuencia en los protocolos SR.

Mecanismo	Uso, Comentarios
Suma de comprobación (Checksum)	Utilizada para detectar errores de bit en un paquete transmitido.
Temporizador	Se emplea para detectar el fin de temporización y retransmitir un paquete, posiblemente porque el paquete (o su mensaje ACK correspondiente) se ha perdido en el canal. Puesto que se puede producir un fin de temporización si un paquete está retardado pero no perdido (fin de temporización prematura), o si el receptor ha recibido un paquete pero se ha perdido el correspondiente ACK del receptor al emisor, puede ocurrir que el receptor reciba copias duplicadas de un paquete.
Número de secuencia	Se emplea para numerar secuencialmente los paquetes de datos que fluyen del emisor hacia el receptor. Los saltos en los números de secuencia de los paquetes recibidos permiten al receptor detectar que se ha perdido un paquete. Los paquetes con números de secuencia duplicados permiten al receptor detectar copias duplicadas de un paquete.
Reconocimiento (ACK)	El receptor utiliza estos paquetes para indicar al emisor que un paquete o un conjunto de paquetes ha sido recibido correctamente. Los mensajes de reconocimiento suelen contener el número de secuencia del paquete o los paquetes que están confirmando. Dependiendo del protocolo, los mensajes de reconocimiento pueden ser individuales o acumulativos.
Reconocimiento negativo (NAK)	El receptor utiliza estos paquetes para indicar al emisor que un paquete no ha sido recibido (NAK) correctamente. Normalmente, los mensajes de reconocimiento negativo contienen el número de secuencia de dicho paquete erróneo.
Ventana, procesamiento en cadena	El emisor puede estar restringido para enviar únicamente paquetes cuyo número de secuencia caiga dentro de un rango determinado. Permitiendo que se transmitan varios paquetes aunque no estén todavía reconocidos, se puede incrementar la tasa de utilización del emisor respecto al modo de operación de los protocolos de parada y espera. Veremos brevemente que el tamaño de la ventana se puede establecer basándose en la capacidad del receptor para recibir y almacenar en buffer los mensajes, o en el nivel de congestión de la red, o en ambos parámetros.

Tabla 3.1 ♦ Resumen de los mecanismos para la transferencia de datos fiable y su uso.

Protocolo TCP

- “Dado que **el protocolo TCP se ejecuta únicamente en los sistemas terminales** y no en los elementos intermedios de la red (routers y switches de la capa de enlace), **los elementos intermedios de la red no mantienen el estado de la conexión TCP**. De hecho, los routers intermedios son completamente inconscientes de las conexiones TCP; los routers ven los datagramas, no las conexiones” [p. 191]
- Proporciona un servicio full-duplex (*pueden enviarse y recibirse paquetes en ambas direcciones a la vez*)

- “TCP empareja cada fragmento de datos del cliente con una cabecera TCP, formando segmentos TCP. Los segmentos se pasan a la capa de red, donde son encapsulados por separado dentro de datagramas IP de la capa de red. Los datagramas IP se envían entonces a la red. Cuando TCP recibe un segmento en el otro extremo, los datos del mismo se colocan en el buffer de recepción de la conexión TCP. La aplicación lee el flujo de datos de este buffer. Cada lado de la conexión tiene su propio buffer de emisión y su propio buffer de recepción” [p. 193]
https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/content/interactiveanimations/flow-control/index.html

¿qué hace un host cuando no recibe los segmentos en orden a través de una conexión TCP?

- “Básicamente, tenemos dos opciones: (1) **el receptor descarta de forma inmediata los segmentos que no han llegado en orden** (lo que, como hemos anteriormente, puede simplificar el diseño del receptor) o (2) **el receptor mantiene los bytes no ordenados y espera a que lleguen los bytes que faltan con el fin de rellenar los huecos**. Evidentemente, esta última opción es más eficiente en términos de ancho de banda de la red, y es el método que se utiliza en la práctica” [p. 196]

Suceso	Acción del receptor TCP
Llegada de un segmento en orden con el número de secuencia esperado. Todos los datos hasta el número de secuencia esperado ya han sido reconocidos.	ACK retardado. Esperar hasta durante 500 milisegundos la llegada de otro segmento en orden. Si el siguiente segmento en orden no llega en este intervalo, enviar un ACK.
Llegada de un segmento en orden con el número de secuencia esperado. Hay otro segmento en orden esperando la transmisión de un ACK.	Enviar inmediatamente un único ACK acumulativo, reconociendo ambos segmentos ordenados.
Llegada de un segmento desordenado con un número de secuencia más alto que el esperado. Se detecta un hueco.	Enviar inmediatamente un ACK duplicado, indicando el número de secuencia del siguiente byte esperado (que es el límite inferior del hueco).
Llegada de un segmento que completa parcial o completamente el hueco existente en los datos recibidos.	Enviar inmediatamente un ACK, suponiendo que el segmento comienza en el límite inferior del hueco.

Tabla 3.2 ♦ Recomendación para la generación de mensajes ACK en TCP [RFC 5681].

- “Un **ACK duplicado** es un ACK que vuelve a reconocer un segmento para el que el emisor ya ha recibido un reconocimiento anterior”
- “En el caso de que **se reciban tres ACK duplicados, el emisor TCP realiza una retransmisión rápida** [RFC 5681], reenviando el segmento que falta antes de que caduque el temporizador de dicho segmento.”
- “Una modificación propuesta para TCP es lo que se denomina reconocimiento selectivo [RFC 2018], que permite a un receptor TCP reconocer segmentos no ordenados de forma selectiva, en lugar de solo hacer reconocimientos acumulativos del último segmento recibido correctamente y en orden. Cuando se combina con la retransmisión selectiva (saltándose la retransmisión de segmentos que ya han sido reconocidos de forma selectiva por el receptor), TCP se comporta como nuestro protocolo SR selectivo. Por tanto, el mecanismo de recuperación de errores de TCP probablemente es mejor considerarlo como **un híbrido de los protocolos GBN y SR.**”

control de flujo

- TCP proporciona un servicio de control de flujo a sus aplicaciones para eliminar la posibilidad de que el emisor desborde el buffer del receptor
- La ventana de recepción, llamada VentRecepcion, se hace igual a la cantidad de espacio libre disponible en el buffer: **VentRecepcion = BufferRecepcion – [UltimoByteRecibido – UltimoByteLeido]**

- ¿Cómo utiliza la conexión la variable VentRecepcion para proporcionar el servicio de control de flujo? El host B dice al host A la cantidad de espacio disponible que hay en el buffer de la conexión almacenando el valor actual de VentRecepcion en el campo ventana de recepción de cada segmento que envía a A.
- la diferencia entre estas dos variables, UltimoByteEnviado – UltimoByteReconocido, es la cantidad de datos no reconocidos que el host A ha enviado a través de la conexión

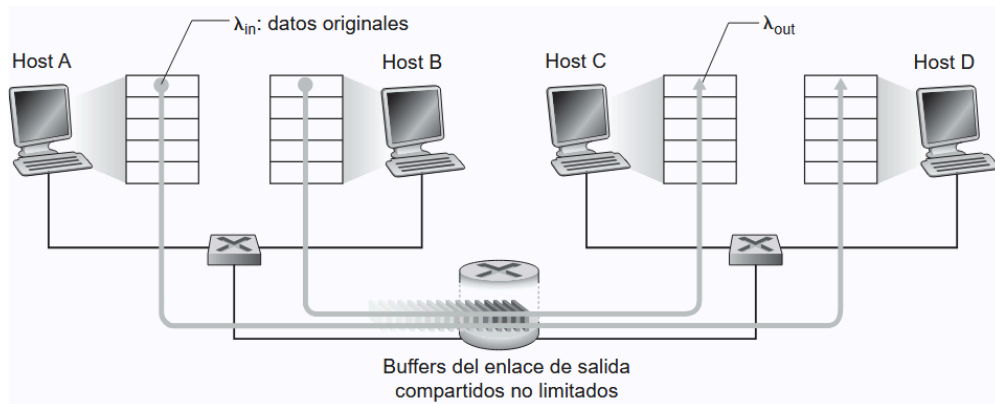
gestión de la conexión (triple handshake y etc)

[Gestión de la conexión TCP.pdf](#)

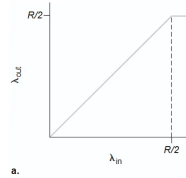
Control de Congestión

- Para tratar la causa de la congestión de la red son necesarios **mecanismos que regulen el flujo de los emisores en cuanto la congestión de red aparezca**

Análisis de la transferencia de datos entre dos redes, cada una con dos hosts, donde los hosts se quieren comunicar con los otros dos hosts de la siguiente red, y donde entre las dos redes hay un router que las conecta, con una capacidad de retención de datos infinita



- Supongamos que host A y host B pertenecen a la Red 1, y que host C y host D pertenecen a la Red 2.
- En el caso de que host A se quiera comunicar con host C, y host B se quiera comunicar con host D, la **tasa de transferencia de datos** dependería de dos cosas:
 1. La **velocidad de transmisión del host A y el host B**, y la **velocidad de transmisión del host C y del host D**
 2. El **ancho de banda del router**, la capacidad máxima del enlace entre las dos redes
- En el mejor de los casos, la tasa de transferencia de datos va a ser de $R/2$, ya que el router en cualquier momento solo puede enviar hasta R paquetes por medio del enlace, y hay dos hosts enviándole paquetes de igual forma → Consultar



- Sin embargo, mientras mas paquetes se envíen al router, y la cantidad de paquetes exceden infinitamente los $R/2$ paquetes que se pueden enviar en un determinado momento, el retardo entre que se envía un paquete y llega a destino crece. → [Consultar](#)

Análisis de una situación parecida a la anterior, pero en donde el router tiene un buffer de capacidad finita, y donde puede ser que los hosts tengan que retransmitir sus paquetes

Costes de una red congestionada

- Los grandes retardos de cola experimentados cuando la tasa de llegada de los paquetes se aproxima a la capacidad del enlace R (ver primer escenario)
- El emisor tiene que realizar retransmisiones para poder compensar los paquetes descartados (perdidos) a causa de un desbordamiento de buffer ([Kurose](#), p. 216 - 218)
- Las retransmisiones innecesarias del emisor causadas por retardos largos pueden llevar a que un router utilice el ancho de banda del enlace para reenviar copias innecesarias de un paquete
- Cuando un paquete se descarta a lo largo de una ruta, la capacidad de transmisión empleada en cada uno de los enlaces anteriores para encaminar dicho paquete hasta el punto en el que se ha descartado termina por desperdiciarse ([Kurose](#), p. 219)

Métodos para controlar la congestión

- **Métodos de control que utilizan asistencia de los routers** → los routers proporcionan una realimentación explícita al emisor y/o receptor informando del estado de congestión en la red
- **Métodos de control de terminal a terminal** → la presencia de congestión en la red tiene que ser inferida por los sistemas terminales basándose únicamente en el comportamiento observado de la red. [requiere especificar indicios de la existencia de congestión en la red](#)

Mecanismo que utiliza TCP

- El método empleado por TCP consiste en que cada emisor limite la velocidad a la que transmite el tráfico a través de su conexión en función de la congestión de red percibida. Si un emisor TCP percibe que en la ruta entre él mismo y el destino apenas existe congestión, entonces incrementará su velocidad de transmisión; por el contrario, si el emisor percibe que existe congestión a lo largo de la ruta, entonces reducirá su velocidad de transmisión

¿cómo limita el emisor TCP la velocidad a la que envía el tráfico a través de su conexión?

- La **ventana de recepción** impone una restricción sobre la velocidad a la que el emisor TCP puede enviar tráfico a la red. Específicamente, la cantidad de datos no reconocidos en un emisor no puede exceder el mínimo de entre VentCongestion y VentRecepcion
- [Si todo sale bien, todos los paquetes se envían y llegan,] TCP interpretará la llegada de estos paquetes ACK como una indicación de que todo está bien (es decir, que los segmentos que están siendo transmitidos a través de la red están siendo entregados correctamente al destino) y empleará esos paquetes de

reconocimiento para **incrementar el tamaño de la ventana de congestión** (y, por tanto, la velocidad de transmisión)

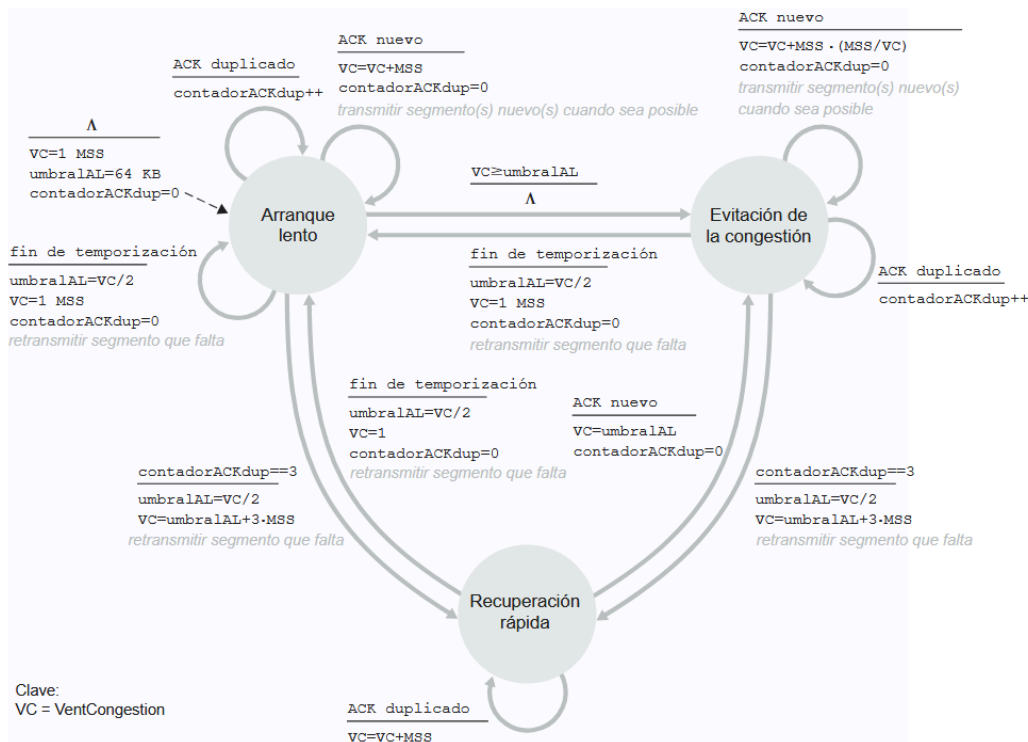
¿cómo percibe el emisor TCP que existe congestión en la ruta entre él mismo y el destino?

- Definamos un **"suceso de pérdida"** en un emisor TCP como el hecho de que se produzca un fin de temporización o se reciban tres paquetes ACK duplicados procedentes del receptor

¿qué algoritmo deberá emplear el emisor para variar su velocidad de transmisión en función de la congestión percibida terminal a terminal?

- la estrategia de TCP para ajustar su velocidad de transmisión consiste en incrementar su velocidad en respuesta a la llegada de paquetes ACK hasta que se produce una pérdida, momento en el que reduce la velocidad de transmisión. El emisor TCP incrementa entonces su velocidad de transmisión para tantear la velocidad a la que de nuevo aparece congestión, retrocede a partir de ese punto y comienza de nuevo a tantear para ver si ha variado la velocidad a la que comienza de nuevo a producirse congestión

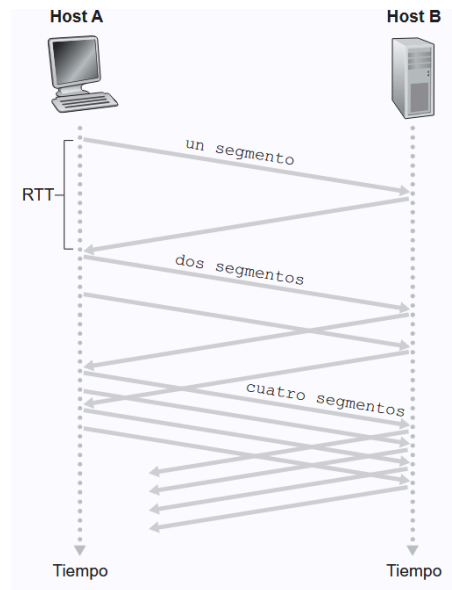
Algoritmo de control de congestión de TCP



arranque lento → repasar

- en el estado de **arranque lento**, el valor de *VentCongestion* se establece en 1 MSS y se incrementa 1 MSS cada vez que se produce el primer reconocimiento de un segmento transmitido
- si se produce un **suceso de pérdida** de paquete (es decir, si hay congestión) señalado por un fin de temporización, el emisor TCP establece el valor de *VentCongestion* en 1 e inicia de nuevo un proceso de arranque lento

- También define el valor de una segunda variable de estado, `umbralAL`, que establece el umbral del arranque lento en $VentCongestion/2$
- cuando el valor de `VentCongestion` es igual a `umbralAL`, la fase de arranque lento termina y las transacciones TCP pasan al modo de evitación de la congestión



evitación de congestión

- Al entrar en el estado de evitación de la congestión, el valor de `VentCongestion` es aproximadamente igual a la mitad de su valor en el momento en que se detectó congestión por última vez

recuperación rápida

- En la fase de recuperación rápida, el valor de `VentCongestion` se incrementa en 1 MSS por cada ACK duplicado recibido correspondiente al segmento que falta y que ha causado que TCP entre en el estado de recuperación rápida

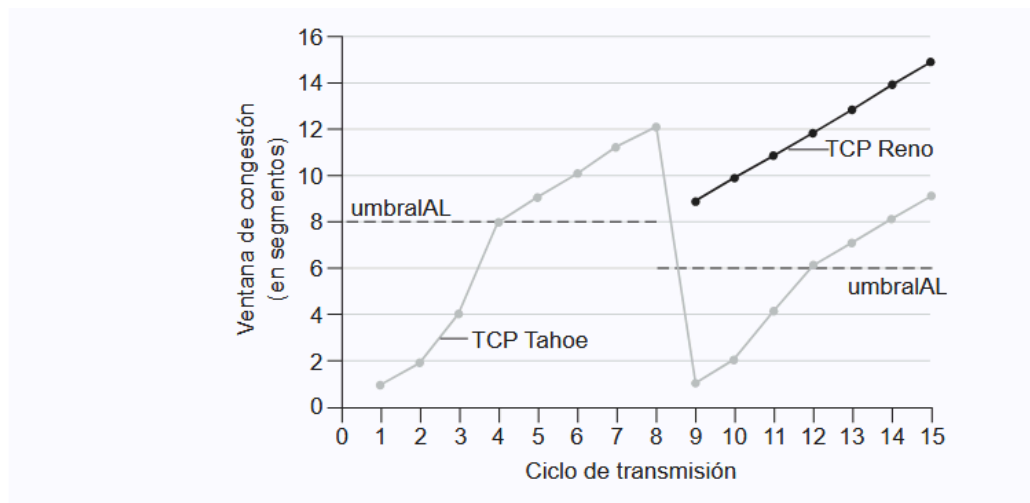


Figura 3.52 ♦ Evolución de la ventana de congestión de TCP (Tahoe y Reno).

TCP Tahoe

- establece incondicionalmente el tamaño de la ventana de congestión en 1 MSS y entra en el estado de arranque lento después de un suceso de pérdida indicado por un fin de temporización o por la recepción de tres ACK duplicados

TCP Reno

- incorpora la recuperación rápida